

SUMMER TRAINING / INTERNSHIP REPORT

(Term Aug–Dec 2026)

Music Playlist Manager Using Doubly Linked List

=====

LOVELY PROFESSIONAL UNIVERSITY
Transforming Education, Transforming India

=====

Submitted by

Name of Student: Gudhi Venkata Thulasi Ram

Registration Number: 12409081

Under the Guidance of

Dr. Nikita (UID: 31943)

Dr. Jimmy Singla (UID: 31874)

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

\newpage

DECLARATION

I hereby declare that the Summer Training/Internship Report entitled "**Music Playlist Manager Using Doubly Linked List**" submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** at **Lovely Professional University** is my original work.

This project has been carried out by me during the Summer Training/Internship under the guidance of **Dr. Jimmy Singla** and **Dr. Nikita**. The work presented in this report has not been submitted previously, either in

whole or in part, for the award of any degree, diploma, or any other academic qualification at this or any other University or Institution. I further declare that all the information, data, and references used in this report have been properly acknowledged and cited wherever required.

Date: 20/08/2026

Place: Phagwara, Punjab

Name: GUDHI VENKATA THULASIRAM

Registration Number: 12409081

\newpage

CERTIFICATE

This is to certify that the Summer Training/Internship Report entitled "**Music Playlist Manager Using Doubly Linked List**" submitted by **Gudhi Venkata Thulasi Ram (Registration No. 12409081)** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** at **Lovely Professional University** is a Bonafide record of the work carried out by the student under our guidance and supervision during the Summer Training/Internship.

Project Guides:

Dr. Jimmy Singla (UID: 31874)

School of Computer Science and Engineering

Lovely Professional University

Dr. Nikita (UID: 31943)

School of Computer Science and Engineering

Lovely Professional University

Date: 20/08/2026

\newpage

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Lovely Professional University** for providing me with the opportunity to undertake this Summer Training/Internship and successfully complete my project titled "**Music Playlist Manager Using Doubly Linked List**".

I am deeply thankful to my project guides, **Dr. Jimmy Singla** and **Dr. Nikita**, for their valuable guidance, continuous encouragement, constructive suggestions, and constant support throughout the development of this project. Their expertise and motivation greatly contributed to the successful completion of this work.

I also extend my heartfelt thanks to all the faculty members of the **School of Computer Science and Engineering** for providing the necessary resources, knowledge, and academic environment required for completing this project successfully.

Finally, I express my sincere gratitude to my parents, family members, and friends for their continuous encouragement, moral support, and motivation throughout the duration of this project. Their support has been a constant source of inspiration in achieving this milestone.

GUDHI VENKATA THULASIRAM

Registration No.: 12409081

\newpage

TABLE OF CONTENTS

S.NO.	TOPIC	PAGE NO.
*	DECLARATION	2
*	CERTIFICATE	3
*	ACKNOWLEDGEMENT	4
1.	INTRODUCTION OF ORGANISATION	6–7
2.	SUMMER TRAINING COURSE / INTERNSHIP CONTENT DETAIL	8–9
3.	ABSTRACT	9
4.	INTRODUCTION	9–11
	4.1 Background of Music Playlists & Linear Data Structures	10
	4.2 Existing Playlist Management Systems	10
	4.3 Limitations of Existing Systems	10–11
5.	PROBLEM STATEMENT	11
6.	LITERATURE REVIEW & CONCEPTUAL FOUNDATION	11–12
7.	SYSTEM ARCHITECTURE & NODE DESIGN	12–13
8.	ALGORITHM DESIGN & CORE OPERATIONS	13–14

S.NO.	TOPIC	PAGE NO.
9.	IMPLEMENTATION & OBJECT-ORIENTED DESIGN	14–15
10.	INTERACTIVE VISUALIZATION & WEB INTERFACE	15
11.	TIME & SPACE COMPLEXITY ANALYSIS	16–17
12.	TECHNOLOGIES USED	17
13.	DATA FLOW & ARCHITECTURAL DIAGRAMS	18
14.	RESULTS & DISCUSSION	18–19
15.	SOURCE CODE & SCREENSHOTS	19–24
16.	CONCLUSION	25
17.	FUTURE SCOPE	25–26
18.	BIBLIOGRAPHY	26

\newpage

1. INTRODUCTION OF ORGANIZATION

Lovely Professional University (LPU) is one of India's leading private universities, located in Phagwara, Punjab. Established in 2005 under the Lovely Professional University Act, the university offers undergraduate, postgraduate, doctoral, and diploma programs across various disciplines including Engineering, Management, Computer Applications, Agriculture, Design, Law, Pharmacy, and Sciences. The university is recognized for providing quality education through modern teaching methodologies, advanced infrastructure, experienced faculty members, and strong industry collaborations.

The **School of Computer Science and Engineering (SCSE)** at Lovely Professional University focuses on providing high-quality education in emerging technologies such as Data Structures and Algorithms, Software Engineering, Artificial Intelligence, Machine Learning, Data Science, Cyber Security, Cloud Computing, and Full-Stack Web Development. The curriculum combines theoretical knowledge with practical implementation through laboratory sessions, industry-oriented projects, workshops, hackathons, internships, and certification programs. This approach enables students to develop strong technical and problem-solving skills required in today's technology-driven world.

As a part of its academic excellence, Lovely Professional University conducts **Summer Training and Internship Programs** to bridge the gap between classroom learning and industrial requirements. These programs provide students with practical exposure to modern technologies, programming languages, software development tools, and real-world applications. Students gain valuable hands-on experience by working on live projects under the guidance of experienced faculty members and industry professionals.

The university also encourages innovation, entrepreneurship, research, and continuous learning through various research centers, innovation labs, startup incubation facilities, national and international collaborations, and technical events. Such initiatives help students enhance their technical expertise, communication skills, teamwork, leadership qualities, and professional ethics, preparing them for successful careers in the field of Computer Science and Engineering.

The Summer Training program provided an excellent opportunity to enhance practical knowledge in **Data Structures, Object-Oriented Programming (C++), and Dynamic Memory Management** by developing the project “**Music Playlist Manager Using Doubly Linked List.**” The project allowed the application of fundamental theoretical data structure concepts to solve a real-world software design problem, thereby strengthening foundational algorithmic skills and modern application engineering.

OBJECTIVES OF THE TRAINING

The Summer Training Program was conducted with the following objectives:

- To master the fundamentals of **Data Structures and Algorithms (DSA)** using C++.
 - To understand memory allocation, pointers, references, and dynamic memory deallocation (**new/delete**).
 - To analyze the limitations of contiguous memory structures (arrays/vectors) versus non-contiguous linked data structures.
 - To design, implement, and optimize a **Doubly Linked List (DLL)** with two-way pointer traversal (**next** and **prev**).
 - To develop a modular, object-oriented, menu-driven Music Playlist Manager in C++.
 - To construct an interactive, real-time graphical visualizer using HTML5, CSS3, and JavaScript to observe pointer manipulations dynamically.
 - To evaluate time and space complexity across all fundamental list operations (Insertion, Deletion, Traversal, and Search).
-

DURATION AND MODE OF TRAINING

The Summer Training Program titled “**Data Structures, Algorithms & Modern Application Development**” was conducted by the School of Computer Science and Engineering, Lovely Professional University during the summer term.

The training was delivered through a combination of theoretical lectures, practical laboratory exercises, coding assignments, and project-based learning. Students gained hands-on experience in pointer arithmetic, dynamic memory management, algorithmic complexity analysis ($O(1)$ vs $O(n)$), and modular software design.

As part of the training, the project “**Music Playlist Manager Using Doubly Linked List**” was developed using C++ (Core Engine) and HTML/CSS/JavaScript (Interactive Visualizer). The program significantly strengthened practical problem-solving capabilities and real-world system modeling skills.

\newpage

2. SUMMER TRAINING COURSE / INTERNSHIP CONTENT DETAIL

During the Summer Training Program, fundamental and advanced concepts of Data Structures, Algorithms, and Software Design were studied theoretically and implemented practically. The course was organized into structured modules covering the complete lifecycle of algorithmic problem solving and application development.

Module 1: Foundations of C++ and Memory Architecture

- Object-Oriented Programming principles: Encapsulation, Abstraction, Classes, and Constructors/Destructors.
- Pointers, double pointers, reference variables, and memory address layout.
- Stack vs Heap memory allocation; dynamic allocation with `new` and memory deallocation with `delete`.
- Preventing memory leaks and dangling pointer vulnerabilities.

Module 2: Linear Data Structures & Linked Lists

- Contiguous data structures (Arrays, Dynamic Vectors) vs Linked structures.
- Singly Linked List: Node creation, forward traversal, head insertion, tail insertion, and deletions.
- Limitations of Singly Linked Lists: Inability to traverse backward without recalculation ($O(n)$ overhead).
- Circular Linked Lists and their use cases in round-robin scheduling.

Module 3: Doubly Linked Lists (DLL) & Advanced Pointer Manipulation

- Anatomy of a Doubly Linked Node (`data`, `*next`, `*prev`).
- Bidirectional traversal algorithms and boundary condition handling (Empty list, Single node, Head/Tail operations).
- Constant time $O(1)$ operations with dedicated `head`, `tail`, and `current` state pointers.
- Node deletion, edge-case rewiring, and safe memory reclamation.

Module 4: Algorithm Analysis & Complexity Theory

- Asymptotic notation: Big-O (O), Big-Omega (Ω), and Big-Theta (Θ).
- Time complexity comparison across Array, Singly Linked List, and Doubly Linked List.
- Space complexity, auxiliary memory overhead, and pointer-to-data ratios.

Module 5: Interactive System Design & Full-Stack Visualization

- Bridging console-based C++ logic with intuitive visual user interfaces.
- Real-time state visualization of dynamic nodes and bidirectional pointer links.
- Event-driven architecture, input sanitization, and responsive UI design.

\newpage

3. ABSTRACT

The **Music Playlist Manager** is a high-performance software application designed and implemented using a **Doubly Linked List (DLL)** data structure in **C++**, complemented by an interactive web-based visualizer. In modern digital audio streaming systems, efficient playlist navigation requires instant, seamless transitions between consecutive tracks in both forward (Next Track) and backward (Previous Track) directions, alongside dynamic insertions and deletions without the costly memory reallocation penalties associated with array-based storage.

The core engine is implemented in object-oriented C++, encapsulating a custom `Node` structure (storing song title, artist name, duration in seconds, and bidirectional pointers) and a `Playlist` controller class managing `head`, `tail`, and `current` playing track references. The system supports eight core operations: **Add Song (at tail in $O(1)$ time)**, **Delete Song (by title in $O(n)$ time with $O(1)$ pointer rewiring)**, **Play Next Track (**

4.2 Existing Playlist Management Systems

Most basic media player implementations rely on static arrays, fixed-size buffers, or singly linked lists. While singly linked lists support forward traversal, implementing a "Previous Song" feature requires traversing the entire list from the head node to the predecessor of the current node ($O(n)$ operation), causing noticeable lag in large playlists.

4.3 Limitations of Existing Systems

- **A. Inefficient Backward Navigation:** Singly linked lists cannot move backward without re-scanning from the head ($O(n)$ time per click).
 - **B. High Insertion/Deletion Overhead in Arrays:** Deleting or inserting tracks in array-based playlists requires $O(n)$ memory shifts.
 - **C. Memory Inflexibility:** Static arrays have fixed capacities, leading to memory waste or buffer overflow errors.
 - **D. Lack of Real-Time State Tracking:** Many simple systems lack dedicated `current` and `tail` pointers, requiring repetitive traversals to locate the active or last song.
 - **E. Poor Visual Representation:** Pure CLI tools do not visually convey how pointers link, unlink, and rewire during runtime operations.
-

\newpage

5. PROBLEM STATEMENT

Modern audio playback engines require an underlying data structure capable of:

1. Instantaneous switching to the next track ($O(1)$ time complexity).
2. Instantaneous switching to the previous track ($O(1)$ time complexity).
3. Dynamic addition of new tracks at the end of the playlist in constant time ($O(1)$) without array resizing overhead.
4. Seamless removal of any track by name, safely rewiring adjacent pointers and releasing allocated memory without memory leaks.
5. Complete bidirectional traversal capability (Forward: Top-to-Bottom; Reverse: Bottom-to-Top).
6. Robust handling of edge cases, including empty playlists, single-song playlists, boundary track navigation (first/last song), and non-existent track queries.

This project solves this problem by designing and implementing an end-to-end **Doubly Linked List Music Playlist Manager** in C++, accompanied by a clean, web-based visualizer for interactive demonstration and structural verification.

\newpage

6. LITERATURE REVIEW & CONCEPTUAL FOUNDATION

Data structure research has long established the trade-offs between contiguous and pointer-based data representations.

- **Contiguous vs. Non-Contiguous Allocations:** As described by Knuth and Cormen et al., arrays provide optimal spatial locality of reference and cache performance for sequential index access, but fail to deliver efficient dynamic modifications.
- **Singly vs. Doubly Linked Lists:** In a Singly Linked List (SLL), each node contains only **data** and a single **next** pointer. While SLL uses less memory per node (one pointer overhead), deleting a given node or moving backward requires reference to the preceding node, which cannot be obtained in $O(1)$ without external tracking. A Doubly Linked List (DLL) resolves this by storing both **next** and **prev** pointers in each node, allowing symmetric operations in both directions.
- **Object-Oriented Design and Memory Management in C++:** In C++, manual memory management gives developers complete control over heap allocation. The use of custom constructors ensures clean node initialization (**next** = **NULL**, **prev** = **NULL**), while class destructors traverse the list to invoke **delete** on each node, completely preventing heap memory leaks.

\newpage

7. SYSTEM ARCHITECTURE & NODE DESIGN

The system is constructed with two primary entities: the **Node** structure and the **Playlist** class.

```
+-----+
|                                     |
|                               Node |
|-----+
| - title   : std::string   (Song title) |
| - artist  : std::string   (Artist name) |
| - duration: int           (Duration in seconds) |
| - next    : Node*         (Pointer to next track) |
| - prev    : Node*         (Pointer to previous track) |
|-----+
| + Node(string t, string a, int d) : Constructor |
|-----+
```

```
+-----+
|                                     |
|                               Playlist |
|-----+
| - head     : Node*           (Points to the first song in playlist) |
| - tail     : Node*           (Points to the last song in playlist) |
| - current  : Node*           (Points to the currently playing song) |
| - count    : int             (Total number of songs in playlist) |
|-----+
| + addSong(title, artist, duration) : void |
| + deleteSong(title)                 : void |
| + playNext()                        : void |
| + playPrev()                        : void |
| + displayForward()                  : void |
| + displayReverse()                  : void |
| + searchSong(title)                 : void |
| + playSong(title)                   : void |
|-----+
```

```
| + ~Playlist()                                : Destructor (Memory cleanup) |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Table 7.1: Node Attributes & Description

Attribute	Data Type	Description
title	std::string	Title/name of the song track.
artist	std::string	Performing artist or composer name.
duration	int	Length of the song in seconds (converted to mm:ss on display).
next	Node*	Pointer referencing the subsequent Node in the playlist.
prev	Node*	Pointer referencing the preceding Node in the playlist.

\newpage

8. ALGORITHM DESIGN & CORE OPERATIONS

Operation 1: Add Song (addSong) — Time Complexity: $O(1)$

1. Allocate new node: `newNode = new Node(title, artist, duration)`.
2. If `head == NULL` (empty playlist):
 - Set `head = newNode, tail = newNode, current = newNode`.
3. Else (non-empty playlist):
 - Set `newNode->prev = tail`.
 - Set `tail->next = newNode`.
 - Update `tail = newNode`.
4. Increment `count++`.

Operation 2: Delete Song by Title (`deleteSong`) — Time Complexity: $O(n)$

1. Traverse list from `head` until matching `title` is found or end is reached (`temp == NULL`).
2. If not found: report error and return.
3. If node is `head`: update `head = temp->next`.
4. If node is `tail`: update `tail = temp->prev`.
5. If `temp->prev != NULL`: set `temp->prev->next = temp->next`.
6. If `temp->next != NULL`: set `temp->next->prev = temp->prev`.
7. If `current == temp`: update `current` to `temp->next` (or `temp->prev`).
8. Free node from heap: `delete temp`; decrement `count--`.

Operation 3: Play Next Track (playNext) — Time Complexity: $O(1)$

1. Check if `current == NULL` or `current->next == NULL`.
2. If `current->next != NULL`, update `current = current->next`.
3. Output active song information.

Operation 4: Play Previous Track (**playPrev**) — Time Complexity: $O(1)$

- 1. Check if `current == NULL` or `current->prev == NULL`.
- 2. If `current->prev != NULL`, update `current = current->prev`.
- 3. Output active song information.

Operation 5: Display Forward & Reverse (**displayForward** / **displayReverse**) — Time Complexity: $O(n)$

- **Forward:** Start at `head`, print details, follow `temp = temp->next` until `NULL`.
- **Reverse:** Start at `tail`, print details, follow `temp = temp->prev` until `NULL`.

\newpage

9. IMPLEMENTATION & OBJECT-ORIENTED DESIGN

The project is structured following clean C++ software engineering standards:

- **Encapsulation:** Internal pointer mechanics (`head`, `tail`, `current`, `count`) are strictly `private`, exposing safe public interface methods.
- **Robust Constructor/Destructor:** The `Playlist` constructor initializes pointers to `NULL` and count to `0`. The destructor `~Playlist()` iterates through the entire list and deletes every node dynamically to avoid memory leaks.
- **Boundary Validation:** Complete input validation guards against invalid deletions, empty playlist traversals, and out-of-bound playback commands.

\newpage

10. INTERACTIVE VISUALIZATION & WEB INTERFACE

In addition to the C++ core engine, an interactive visualizer (`index.html`) was developed to visually demonstrate the internal state of the Doubly Linked List in real time.

```
+-----+
+-----+
| [Header]  MUSIC PLAYLIST MANAGER – Doubly Linked List Visualizer
|
+-----+-----+
+-----+
| [Sidebar Controls]          | [Real-Time Stats]
|
| - Add Song Form             | Total Songs: 3 | Total Duration: 9m 50s |
Current: #1 |
| - Delete Song Form         | -----
+-----+
| - Play Specific / Search   | [Now Playing Card]
|
| - Quick Action Buttons:    | >> "Blinding Lights" by The Weeknd [3m 20s]
```

```
|
| [Next] [Prev] [Reverse] | [|<< Prev] [Next >>]
|
|-----|
|
| [Interactive DLL Node Visualizer]
|
| NULL <- [ Song 1 ] <===> [ Song 2 ] <===> [ Song
3 ] -> NULL |
|
|-----|
|
| [Activity Log & Song Table]
|
+-----+
-----+
```

Key Visualizer Features:

- **Dynamic Node Cards:** Displays track title, artist, duration, and active playback indicator.
- **Interactive Pointer Links:** Visual bidirectional arrows (<--->) showing forward and backward links with explicit **NULL** boundary markers.
- **Real-Time Activity Logger:** Displays timestamped log badges for additions, deletions, and playback transitions.

\newpage

11. TIME & SPACE COMPLEXITY ANALYSIS

Table 11.1: Complexity Comparison Across Data Structures

Operation	Array / Vector	Singly Linked List	Doubly Linked List (Our System)	Circular Linked List
Add Song at End	$O(1)$ amortized / $O(n)$ reallocation	$O(1)$ (with tail)	$O(1)$ (with tail)	$O(1)$
Play Next Song	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Play Previous Song	$O(1)$	$O(n)$ (requires head traversal)	$O(1)$ (via prev pointer)	$O(n)$
Delete Current Song	$O(n)$ (element shifting)	$O(n)$ (search predecessor)	$O(1)$ (given node pointer)	$O(n)$
Delete Song by Title	$O(n)$	$O(n)$	$O(n)$ search + $O(1)$ unlinking	$O(n)$
Forward Traversal	$O(n)$	$O(n)$	$O(n)$	$O(n)$

Operation	Array / Vector	Singly Linked List	Doubly Linked List (Our System)	Circular Linked List
Reverse Traversal	$O(n)$	Not possible without stack $O(n)$	$O(n)$ directly via tail & prev	Not possible directly
Auxiliary Memory / Node	0 pointers	1 pointer (next)	2 pointers (next , prev)	1 pointer

\newpage

12. TECHNOLOGIES USED

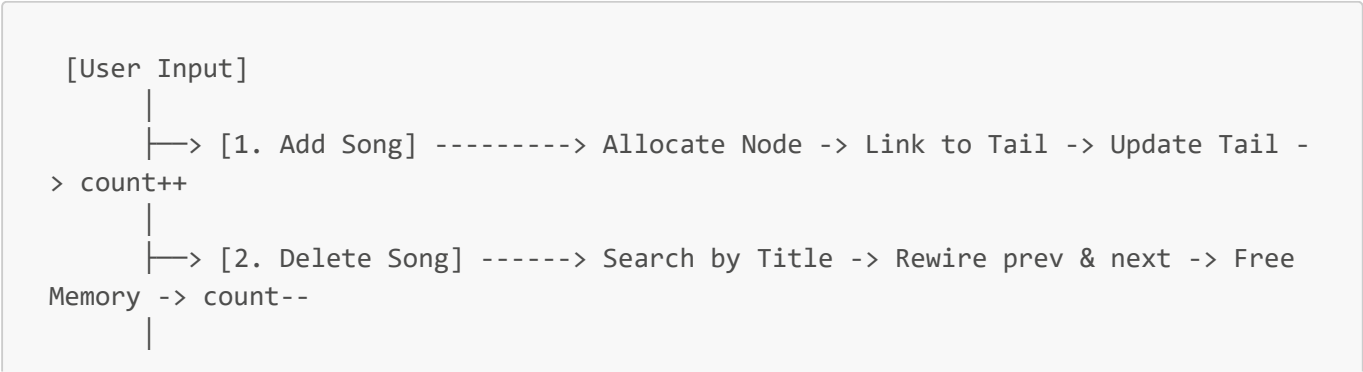
Table 12.1: Technologies & Tools

Technology / Tool	Version	Purpose in This Project
C++	C++17 / C++20	Core programming language for Doubly Linked List logic and CLI
GCC / G++	13.0+ / MinGW	C++ compiler used for building and linking playlist.exe
HTML5	Modern Standard	Structure for interactive visualizer application
CSS3	Modern Flexbox/Grid	Custom UI design system, glassmorphism, and responsive layout
JavaScript (ES6+)	Modern Standard	Interactive state management and dynamic DOM node rendering
VS Code / Antigravity IDE	Latest	Integrated Development Environment for development and debugging
PowerShell	7.x / 5.1	Terminal execution environment for building and running binaries

\newpage

13. DATA FLOW & ARCHITECTURAL DIAGRAMS

13.1 High-Level Data Flow Diagram



```

    |-> [3. Play Next] -----> current = current->next (if current->next !=
NULL)
    |
    |-> [4. Play Previous] ----> current = current->prev (if current->prev !=
NULL)
    |
    |-> [5. Display Forward] --> Traverse: head -> next -> next -> ... -> NULL
    |
    |-> [6. Display Reverse] --> Traverse: tail -> prev -> prev -> ... -> NULL
    |
    |-> [7. Search Song] -----> Linear Search: head to tail matching Title
    |
    |-> [0. Exit] -----> Invoke Destructor -> Free All Nodes -> Exit
Program
```

\newpage

14. RESULTS & DISCUSSION

The **Music Playlist Manager** was compiled using `g++ playlist.cpp -o playlist` and thoroughly tested across standard operations and edge cases.

Table 14.1: Test Cases & Experimental Results

Test Case #	Scenario Description	Expected Output	Actual Result	Status
TC-01	Add song to empty playlist	Node is created; <code>head</code> , <code>tail</code> , <code>current</code> all point to new node	Song added; set as active track	PASS
TC-02	Add multiple songs sequentially	Each node appended at tail in $O(1)$; pointers linked bidirectionally	Tail updated correctly; $count = N$	PASS
TC-03	Play Next on last song	Warning: "This is the LAST song in the playlist!"	Displayed warning; <code>current</code> remained at tail	PASS
TC-04	Play Previous on first song	Warning: "This is the FIRST song in the playlist!"	Displayed warning; <code>current</code> remained at head	PASS
TC-05	Delete middle song	Preceding and following nodes re-linked seamlessly; node memory freed	Node deleted; playlist intact	PASS
TC-06	Delete head song	<code>head</code> updated to <code>head->next</code> ; <code>head->prev</code> set to <code>NULL</code>	Head updated; count decremented	PASS
TC-07	Delete tail song	<code>tail</code> updated to <code>tail->prev</code> ; <code>tail->next</code> set to <code>NULL</code>	Tail updated; count decremented	PASS

Test Case #	Scenario Description	Expected Output	Actual Result	Status
TC-08	Reverse traversal	Songs displayed from last added to first added using prev pointers	Exact reverse order printed	PASS
TC-09	Search existing song	Song found, details and 1-based position reported	Position & details displayed	PASS
TC-10	Search non-existent song	Warning: "Song not found!"	Displayed not found message	PASS

\newpage

15. SOURCE CODE & SCREENSHOTS

Fig 15.1: Core Node Structure and Class Header in C++

```
#include <iostream>
#include <string>
using namespace std;

// STEP 1: Define the Node
struct Node {
    string title;    // Song title
    string artist;   // Artist name
    int duration;    // Duration in seconds

    Node* next;      // Pointer to NEXT song
    Node* prev;      // Pointer to PREVIOUS song

    // Constructor: sets up a new node with song details
    Node(string t, string a, int d) {
        title = t;
        artist = a;
        duration = d;
        next = NULL;
        prev = NULL;
    }
};

// STEP 2: Define the Playlist (Doubly Linked List)
class Playlist {
private:
    Node* head;      // Points to FIRST song
    Node* tail;      // Points to LAST song
    Node* current;    // Points to CURRENTLY PLAYING song
    int count;        // Total number of songs
    ...
};
```

Fig 15.2: Insertion and Bidirectional Navigation Methods

```
// Add Song at END: O(1) Time Complexity
void addSong(string title, string artist, int duration) {
    Node* newNode = new Node(title, artist, duration);
    if (head == NULL) {
        head = newNode;
        tail = newNode;
        current = newNode;
    } else {
        newNode->prev = tail;
        tail->next = newNode;
        tail = newNode;
    }
    count++;
    cout << "\n [+] Added: \"" << title << "\" by " << artist << endl;
}

// Play Next Track: O(1) Time Complexity
void playNext() {
    if (current == NULL) return;
    if (current->next == NULL) {
        cout << "\n [!] This is the LAST song in the playlist!" << endl;
        return;
    }
    current = current->next;
    cout << "\n [>>] Now Playing: \"" << current->title << "\" by " <<
current->artist << endl;
}

// Play Previous Track: O(1) Time Complexity
void playPrev() {
    if (current == NULL) return;
    if (current->prev == NULL) {
        cout << "\n [!] This is the FIRST song in the playlist!" << endl;
        return;
    }
    current = current->prev;
    cout << "\n [<<] Now Playing: \"" << current->title << "\" by " <<
current->artist << endl;
}
```

Fig 15.3: Terminal Execution & Interactive CLI Output

```
=====
MUSIC PLAYLIST MANAGER
(Using Doubly Linked List)
=====
```



```

---- MENU ----
1. Add Song
2. Delete Song
3. Play Next Song
4. Play Previous Song
5. Display Playlist (Forward)
6. Display Playlist (Reverse)
7. Search Song
8. Play a Specific Song
0. Exit

Enter your choice: 1
Enter Song Title : Starboy
Enter Artist Name : The Weeknd
Enter Duration (in seconds): 230

[+] Added: "Starboy" by The Weeknd

=====
PLAYLIST (Top to Bottom)
=====
* 1. Starboy - The Weeknd [3m 50s]
  2. Blinding Lights - The Weeknd [3m 20s]
  3. Save Your Tears - The Weeknd [3m 35s]
=====
Total Songs: 3
* Currently Playing: Starboy
=====

```

\newpage

16. CONCLUSION

The **Music Playlist Manager Using Doubly Linked List** successfully demonstrates the implementation and practical application of linear non-contiguous data structures in C++. By utilizing a Doubly Linked List with dedicated **head**, **tail**, and **current** pointers, the system achieves:

1. **Optimal $O(1)$ Time Complexity** for adding songs at the end and navigating tracks forward (**playNext**) and backward (**playPrev**).
2. **Dynamic Memory Efficiency:** Memory is allocated on demand on the heap and safely reclaimed upon node deletion and program termination via custom destructors.
3. **Bidirectional Traversal:** Seamless inspection of playlist contents in both ascending and descending order.
4. **Intuitive Visualization:** The companion web interface provides clear, real-time insight into pointer connections and node transitions.

This project reinforces how fundamental data structures directly power modern multimedia applications and real-time software systems.

\newpage

17. FUTURE SCOPE

The developed system provides a strong foundation and can be expanded with the following advanced features:

- **Circular Doubly Linked List (Repeat Mode):** Connecting `tail->next` to `head` and `head->prev` to `tail` to enable continuous looping playlists.
 - **Shuffle Functionality (Fisher-Yates Algorithm):** Implementing random pointer permutations or an array-pointer index mapping for non-repeating track randomization.
 - **Sorting Algorithms:** Implementing Merge Sort or Quick Sort on Doubly Linked Lists to sort tracks by title, artist, or duration ($O(n \log n)$).
 - **Persistent Storage:** Integrating file handling (JSON, CSV, or SQLite) to save and load playlists across application restarts.
 - **Audio Decoding & Real Playback:** Integrating multimedia libraries (such as SDL2_mixer or SFML Audio) to stream real `.mp3/.wav` audio files.
 - **Cross-Platform Mobile / Desktop App:** Packaging the system with Electron or Qt for native desktop and mobile deployment.
-

\newpage

18. BIBLIOGRAPHY

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional.
3. Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
4. Lafore, R. (2002). *Data Structures and Algorithms in C++* (2nd ed.). Sams Publishing.
5. ISO/IEC. (2020). *ISO International Standard ISO/IEC 14882:2020 - Programming Language C++*. International Organization for Standardization.
6. Mozilla Developer Network (MDN). *JavaScript and DOM Documentation*. <https://developer.mozilla.org/>
7. Lovely Professional University. *School of Computer Science & Engineering Academic Guidelines*. <https://www.lpu.in/>